

ПРОЕКТ:
ЗАДАЧА КЛАССИФИКАЦИИ ОТТОКА
КЛИЕНТОВ БАНКА

Пример данных в исходной таблице

id	CustomerId	Surname	Credit Score	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCredit Card	IsActive Member	Estimated Salary	Exited
0	15674932	Okwudilichu	668	France	Male	33	3	0	2	1	0	181450	0
1	15749177	Okwudiliolis	627	France	Male	33	1	0	2	1	1	49503.5	0
2	15694510	Hsueh	678	France	Male	40	10	0	2	1	0	184866.7	0
3	15741417	Kao	581	France	Male	34	2	148882.54	1	1	1	84560.88	0
4	15766172	Chiemenam	716	Spain	Male	33	5	0	2	1	1	15068.83	0
5	15771669	Genovese	588	Germany	Male	36	4	131778.58	1	1	0	136024.3	1
6	15692819	Ch'ang	593	France	Female	30	8	144772.69	1	1	0	29792.11	0
7	15669611	Chukwuebuk	678	Spain	Male	37	1	138476.41	1	1	0	106851.6	0
8	15691707	Manna	676	France	Male	43	4	0	2	1	0	142917.1	0
9	15591721	Cattaneo	583	Germany	Male	40	4	81274.33	1	1	1	170843.1	0
10	15635097	Fokine	599	Germany	Female	27	6	161801.47	2	1	0	109184.2	0

Обозначения:

Данные к удалению из датасета

Данные готовы для энкодинга

Строковые значения, необходимо преобразовать

Target, искомый признак

Предобработка и очистка данных 1

- ✓ Проверил дубликаты в исходных данных:
- ✓ Проверка пропусков в данных
- ✓ Столбец CustomerId имеет повторы айди клиентов - это аномалия, но столбец все равно к удалению
- ✓ Удалил столбцы
- ✓ После удаления столбцов, перепроверка дубликатов и удаление. Количество дубликатов не значительно в общем объеме данных.

```
entire row duplicates: 0  
Duplicates in the column: "id" - NO
```

```
data gaps in cells - NO
```

```
duplicate values example:
```

```
[15572767, 15806918, 15757931, 15581261, 15763612,  
15720353, 15763515, 15592089, 15612494, 15660385,  
15737408, 15717191, 15812518, 15650424, 15654495,
```

```
COLUMNS_DROP = ['id', 'CustomerId', 'Surname']
```

```
total dataset rows:165034  
entire row duplicates: 123  
Removed 123 duplicated rows
```

```
rows:164911  columns:11  
entire row duplicates: 0  
-----
```

Предобработка и очистка данных 2

Необходимо преобразовать строковые столбцы в цифровые значения, для этого:

✓ Проверил данные в `Geography`

Количество категорий в обоих столбцах – небольшое. Можно применять one hot encoding или label encoding или просто перечислить категории как enum

✓ Проверил данные в `Gender`

```
column_stats_table(df_filtered, 'Geography')  
[10]
```

	value	count	percentage
0	France	94215	57.088236
1	Spain	36213	21.942751
2	Germany	34606	20.969012

```
column_stats_table(df_filtered, 'Gender')  
[11]
```

	value	count	percentage
0	Male	93150	56.442915
1	Female	71884	43.557085

Предобработка и очистка данных 3

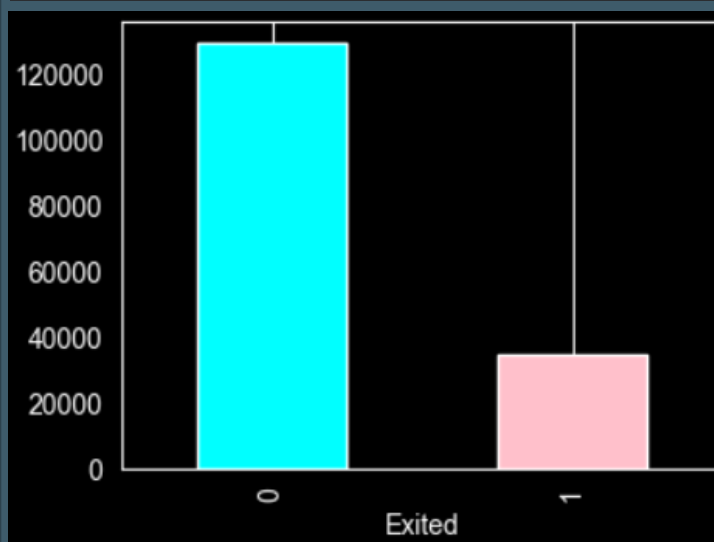
- ✓ Проверил распределение данных в таргете

Соотношение в исходных данных получилось ~79/21
Необходимо будет учесть дисбаланс в моделях и
применить StratifiedKFold для сохранения такой же
пропорции в тренировочных/проверочных фолдах

```
column_stats_table(df_filtered, 'Exited')  
[16]
```

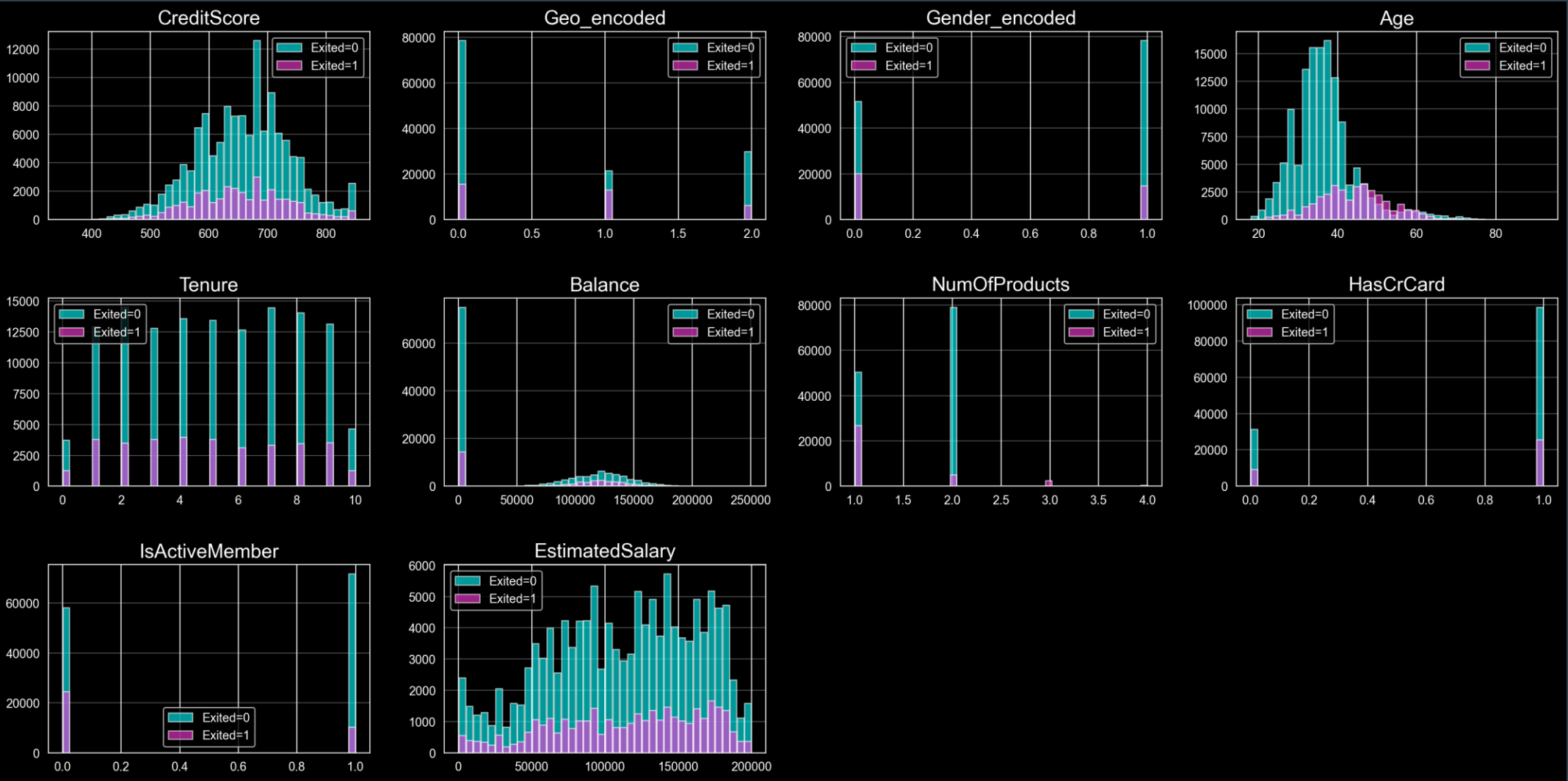
	value	count	percentage
0	0	130113	78.840118
1	1	34921	21.159882

```
colors = ['cyan', 'pink']  
df_filtered['Exited'].value_counts().plot.bar(figsize=(4, 3), color=colors)  
plt.grid(axis='y')  
target_col = 'Exited'
```



Предобработка и очистка данных 4

Общая визуализация распределений сырых данных после label encoding



Модели 1

Проведены эксперименты на нескольких моделях с перебором разных скейлеров:

```
scalers = {  
    'standardScaler': StandardScaler(),  
    'minmaxScaler': MinMaxScaler(),  
    'robustScaler': RobustScaler()  
}
```

```
# =====  
# BLOCK 20: LOGISTIC REGRESSION  
# =====
```

```
# =====  
# BLOCK 21: DECISION TREE  
# =====
```

```
# =====  
# BLOCK 22: TRAIN LIGHTGBM  
# =====
```

```
# =====  
# BLOCK 23: TRAIN CATBOOST  
# =====
```

Модели 2

Результаты с лучшими метриками логируются в csv. Через 50 прогонов понял, что StandardScaler оказался лучше. По итогу, дальше использовал только его для экспериментов. Пример таблицы с логами:

date_time	model_name	scaler	accuracy	recall	precision	f1_score	train_time_ seconds	model_ version
2026-05-31T20:53:47+03:00	LogisticRegression_C1_L2	robustScaler	0.826	0.3447	0.6734	0.4559	3.5477	0.2.1
2026-05-31T21:20:54+03:00	LogisticRegression_C1_L2	standardScaler	0.826	0.3447	0.6734	0.456	20.309	0.3.2
2026-05-31T21:21:11+03:00	LogisticRegression_C1_L2	minmaxScaler	0.8259	0.3438	0.6738	0.4553	17.27	0.3.2
2026-05-31T21:21:30+03:00	LogisticRegression_C1_L2	robustScaler	0.826	0.3447	0.6735	0.456	18.552	0.3.2
2026-05-31T22:00:34+03:00	LogisticRegression_C0.001_ElasticNet (0.8)_balanced	standardScaler	0.7429	0.7334	0.4361	0.5469	23.591	0.3.2
2026-05-31T22:01:00+03:00	LogisticRegression_C0.001_ElasticNet (0.8)_balanced	minmaxScaler	0.7185	0.7271	0.4074	0.5222	25.263	0.3.2
2026-05-31T22:15:32+03:00	LogisticRegression_C0.0005_ElasticN et(0.7)_balanced	standardScaler	0.7442	0.7332	0.4377	0.5482	21.604	0.3.2
2026-05-31T22:15:54+03:00	LogisticRegression_C0.0005_ElasticN et(0.7)_balanced	minmaxScaler	0.6945	0.7178	0.382	0.4986	21.96	0.3.2
2026-05-31T22:16:13+03:00	LogisticRegression_C0.0005_ElasticN et(0.7)_balanced	robustScaler	0.7497	0.7275	0.4441	0.5515	19.578	0.3.2
2026-06-09T14:32:49+03:00	LogisticRegression_C0.0005_ElasticN et(0.7)_balanced	standardScaler	0.7442	0.7332	0.4377	0.5482	22.576	0.3.3
2026-06-09T14:33:13+03:00	LogisticRegression_C0.0005_ElasticN et(0.7)_balanced	minmaxScaler	0.6945	0.7178	0.382	0.4986	24.261	0.3.3

Модели 3

Логистическая регрессия показала совсем плохие результаты

Decision tree – получилось получше, если очень точно подбирать настройки (что может привести к переобучению)

LightGBM и CatBoost оказались лучшие, поэтому сосредоточился на них и занялся фича инжинирингом

Перепроверял все модели с новыми комбинациями по фичам: LightGBM и CatBoost стабильно лучшие результаты

```
# Grid parameters declared as variables
learning_rates = [0.086] # [0.086, 0.087, 0.88]
max_depth_options = [ 6] # [5, 6, 7]
num_leaves_options = [15, 16] # [7, 9, 12, 14, 15, 16, 17, 18]
n_estimators_options = [ 160] # [140, 150, 160, 170]
imbalance_weight_options = [1.95, 2.0] # [1.6, 1.8, 1.9, 1.95, 2.0, 2.1]

learning_rates = [0.048] # [0.05, 0.06, 0.07, 0.08, 0.09]
depths = [7, 8] # [7, 8, 9, 10]
iterations_options = [420] # [300, 400, 800]
l2_leaf_reg_options = [ 7] # [4, 5, 6, 7, 8]
imbalance_weight_options = [1.8] # [1.5, 1.7, 1.8, 2.0, 2.5, 3.0]
```

```
# best on expanded df: lr=0.060, depth=8, iterations=400, l2_reg=4, scale_pos_weight=2.0 Best F1=0.6647
```

```
# best on expanded df: lr=0.048, depth=7, iterations=420, l2_reg=7, scale_pos_weight=1.8 Best F1=0.6668 (no age cate
```

Фича инжиниринг 1

✓ Перепробовал разные способы энкодинга исходных параметров

✓ Сделал отдельную копию датасета, по советам ai добавил комбинации как новые фичи

```
df_expanded = df_alt_features.copy()

# 1. Balance-to-Salary Ratio (Identifies high-earners with empty accounts)
df_expanded['Balance_Salary_Ratio'] = df_expanded['Balance'] / (df_expanded['EstimatedSalary'] + 1)

# 2. Product Utilization Velocity (How fast they adopt products relative to time spent)
df_expanded['Products_Per_Year'] = df_expanded['NumOfProducts'] / (df_expanded['Tenure'] + 1)

# 3. Financial Activity Index (Combines credit card ownership and active membership status)
df_expanded['Financial_Activity'] = df_expanded['HasCrCard'] + df_expanded['IsActiveMember']

# 4. Age and Tenure Interaction (Differentiates young new accounts from older loyal accounts)
df_expanded['Tenure_Age_Interaction'] = df_expanded['Age'] * df_expanded['Tenure']
```

Фича инжиниринг 2

В итоге, получил прирост по метрикам благодаря:

- ✓ ONE для категориальных данных (Geography, Gender),
- ✓ StandardScaler,
- ✓ 4 дополнительные фичи
- ✓ модели CatBoost и LightGBM

Интересные результаты получились при добавлении 'imbalance_weight_options' в перебор параметров

Через одну эту переменную можно управлять насколько агрессивно модель будет результировать в таргет.

Учитывая, что речь идет о потере клиентов банком, кажется, что лучше иметь больший recall, потратиться на удержание клиента, чем потерять клиента

Geo_France	Geo_Germany	Geo_Spain	Gender_Female	Gender_Male
1	0	0	0	1
1	0	0	0	1

Balance_Salary_Ratio	Products_Per_Year	Financial_Activity	Age_Tenure_Interaction
0.000000	0.500000	1	99.0
0.000000	1.000000	2	33.0

```
imbalance_weight_options = [1.5 , 2.0] #[2.0, 2.5 ,3.0]
```

```
# Param dictionary for random search
param_distributions = {
    "learning_rate": learning_rates,
    "depth": depths,
    "iterations": iterations_options,
    "l2_leaf_reg": l2_leaf_reg_options,
    "scale_pos_weight": imbalance_weight_options
```

Фича инжиниринг 3

Пример неудачной фичи (возраст на категории), конкретно эта фича уменьшала recall и precision

```
bins = [0, 16, 24, 37, 46, 61, np.inf]
labels = ['0-15', '17-23', '24-36', '37-45', '46-60', '61+']

# new category column
df_expanded["Age_Category"] = pd.cut(df_expanded["Age"], bins=bins, labels=labels, right=False)

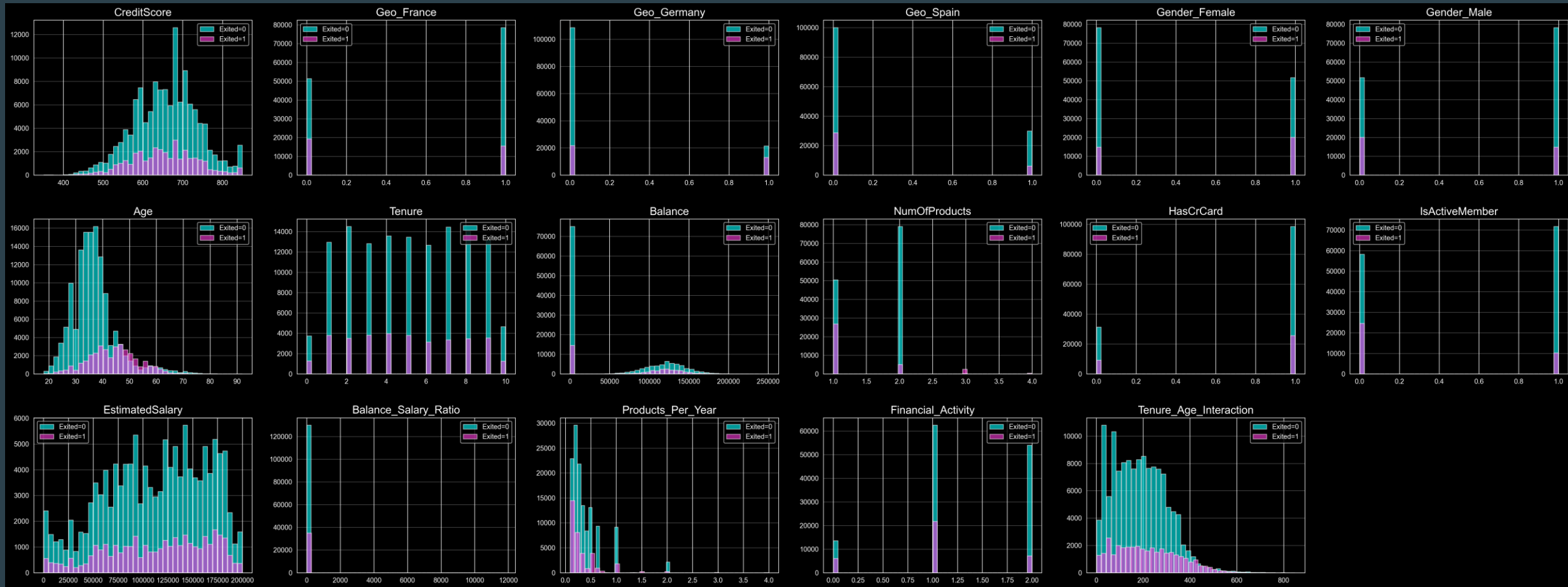
# ohe for age cats
age_cols = []
age_cols = df_alt_features.columns[df_alt_features.columns.str.startswith('age_')]
if age_cols.size > 0:
    df_expanded.drop(columns=age_cols, inplace=True)

# join ohe columns, drop the original column after that
idx = df_expanded.columns.get_loc('Age_Category')
age_ohe = pd.get_dummies(df_expanded['Age_Category'], prefix='age_', dtype=int)

for i, col_name in enumerate(age_ohe.columns):
    df_expanded.insert(idx + 1 + i, col_name, age_ohe[col_name])

df_expanded.drop(columns=['Age', 'Age_Category', 'age__0-15'], inplace=True)
```

Визуализация итогового набора столбцов



Настройки тренировочных и тестовых данных

Разделение тренировочных и тестовых данных

```
cv = StratifiedKFold(n_splits=7, shuffle=True, random_state=555)
```

```
Y = df_expanded['Exited']
```

```
X_raw = df_expanded.drop(['Exited'], axis=1)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_raw, Y, test_size=0.2, random_state=555, stratify=Y)
```

Итоги 2 LightGBM

Сравнение метрик на тренировочных и тестовых данных (для LightGBM)

Тренировочные данные

```
Train time: 4.6453 seconds
Accuracy: 0.8542 (+/- 0.0037)
Recall: 0.6882
Precision: 0.6463
F1-Score: 0.6665
```

Тестовые данные

```
Test Accuracy: 0.8496
Test Recall: 0.6776
Test Precision: 0.6359
Test F1-Score: 0.6561
```

Тест данные – 20% (32983 строки), итог:

True negative	False Positive
[[23292	2709]
[2251	4731]]
False negative	True Positive

Проверка разницы train и test

```
Cross-Validation F1 (average): 0.6665
Test Set F1: 0.6561
Difference (CV - Test): 0.0105
```

Итоги 3 CatBoost

Сравнение метрик на тренировочных и тестовых данных (для CatBoost)

Тренировочные данные

```
Train time: 35.3894 seconds
Accuracy: 0.8585 (+/- 0.0060)
Recall: 0.6691
Precision: 0.6646
F1-Score: 0.6668
```

Тестовые данные

```
Test Accuracy: 0.8536
Test Recall: 0.6568
Test Precision: 0.6533
Test F1-Score: 0.6550
```

Тест данные – 20% (32983 строки), итог:

True negative	False Positive
[[23567	2434]
[2396	4586]]
False negative	True Positive

Проверка разницы train и test

```
Cross-Validation F1 (average): 0.6668
Test Set F1: 0.6550
Difference (CV - Test): 0.0118
```

Вывод:

- ✓ Лучшие результаты у **LightGBM** и **CatBoost**
- ✓ Разница метрик на **train** и **test** минимальные
- ✓ Баланс между **Recall** и **Precision** – приемлемый

Итоги 4

- ✓ Провел полный рефактор кода на версии 0.5
- ✓ сделал логирование метрик и комбинаций
- ✓ учел лучшие фишки и комбинации настроек
- ✓ отметил особое влияние факторов
- ✓ Оценил результаты моделей по метрикам Accuracy, Recall, Precision, F1 score

Лучшие результаты по моделям

model_name	scaler	accuracy	recall	precision	f1_score	train_time_seconds	model_version
CatBoost_lr0.046_depth8_iter500_l2_leaf_reg_options6_scale_pos_weight2.0	StandardScaler	0.8516	0.6825	0.6403	0.6607	51.6443	0.5.6
CatBoost_lr0.046_depth8_iter500_TEST	StandardScaler	0.8467	0.6707	0.6295	0.6495	8.0682	0.5.6
CatBoost_lr0.048_depth7_iter420_l2_leaf_reg_options7_scale_pos_weight1.8	StandardScaler	0.8585	0.6691	0.6646	0.6668	35.9432	0.5.6
CatBoost_lr0.048_depth7_iter420_TEST	StandardScaler	0.8536	0.6568	0.6533	0.655	5.6591	0.5.6
LightGBM_lr0.087_leaves16_estimators160_max_depth6_scale_pos_weight2.0	StandardScaler	0.8486	0.6921	0.6294	0.6593	5.543	0.5.6
LightGBM_lr0.087_leaves16_estimators160_TEST	StandardScaler	0.8441	0.684	0.6193	0.6501	0.489	0.5.6
LightGBM_lr0.086_leaves15_estimators160_max_depth6_scale_pos_weight1.95	StandardScaler	0.8542	0.6882	0.6463	0.6665	5.291	0.5.6
LightGBM_lr0.086_leaves15_estimators160_TEST	StandardScaler	0.8496	0.6776	0.6359	0.6561	0.5625	0.5.6